Q1: What is the output?

```
int a = 10;
int *p = &a;
printf("%d", *p++);
```

Q2: What is the output of the following code?

```
int arr[] = {10, 20, 30};
int *p = arr;
printf("%d", *++p);
```

Q3: What does *p++ vs (*p)++ vs *++p vs ++*p do?

Q4: What is the result of adding 1 to a pointer?

Q5: Is pointer subtraction valid?

```
int arr[5] = {1, 2, 3, 4, 5};
int *p1 = &arr[1];
int *p2 = &arr[4];
printf("%td", p2 - p1);
```

Q6: Can you add two pointers?

Q7: What is a dangling pointer?

Q8: What is a wild pointer?

Q9: What is a NULL pointer vs a Void pointer?

Q10: What is the size of void * on a 64-bit architecture?

Q11: What is the output?

```
int a = 0x12345678;
char *p = (char *)&a;
printf("0x%x", *p);
```

Q12: Is &p identical to p for a given pointer variable p?

Q13: What happens when you assign an integer directly to a pointer?

```
int *p = (int *)0x1000;
```

Q14: What is ptrdiff_t and which standard header defines it?

Q15: What is the output?

```
int arr[3] = {1, 2, 3};
printf("%d", 0[arr]);
```

Q16: What is const int *p vs int * const p?

Q17: What does const int * const p represent?

Q18: What is strict aliasing in C?

Q19: Why does casting int * to char * not violate strict aliasing?

Q20: What is the output?

```
char *s = "Hello";
s[0] = 'h';
```

Q21: What is the difference between arr and &arr for int arr[5]?

Q22: What is the output?

```
int arr[5] = {1, 2, 3, 4, 5};

printf("%p, %p\n", arr + 1, &arr + 1);
```

Q23: What does int (*p)[5] mean?

Q24: What does int *p[5] mean?

Q25: How do you declare a pointer to a function taking int and returning char?

Q26: What is the output?

```
int arr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

printf("%d", *(*(arr + 1) + 2));
```

Q27: How are 2D arrays laid out in memory in C?

Q28: What is the output?

```
int a[5] = {10, 20, 30, 40, 50};
int *p = (int *)(&a + 1);
printf("%d", *(p - 1));
```

Q29: What happens when passing a 2D array to a function like void foo(int arr[][3])?

Q30: What is a Variable Length Array (VLA) pointer?

Q31: Can sizeof evaluate pointer arithmetic at runtime?

```
int arr[5] = {1, 2, 3, 4, 5};
int *p = arr;
printf("%zu", sizeof(*p++));
```

Q32: What is the output?

```
char arr[] = "Hello";
char *p = "Hello";
printf("%zu, %zu", sizeof(arr), sizeof(p));
```

Q33: How do you dynamically allocate a 2D array in contiguous memory?

Q34: What is the decay rule for arrays in C?

Q35: What is the output?

```
int a[3] = {1, 2, 3};
int *p = a;
printf("%d", p[1]);
```

Q36: Is &arr[0] identical to arr?

Q37: Why is sizeof(arr) / sizeof(arr[0]) invalid inside a function receiving an array parameter?

Q38: What does int (*p())[5] represent?

Q39: What does int (*x[10])(int) represent?

Q40: What is the output?

```
int a[2][2] = {1, 2, 3, 4};
printf("%d", **a);
```

Q41: What is Structure Padding?

Q42: What is Structure Packing?

Q43: What is the size of this struct on a 32-bit machine?

```
struct Test {
        char a;
        int b;
        char c;
};
```

Q44: What is a Self-Referential Structure?

Q45: Why cannot a structure contain a direct instance of itself?

Q46: What is the output?

```
struct Data {
    int x;
    int y;
};
struct Data d = {10, 20};
```

```
int *p = (int *)&d;
printf("%d", *(p + 1));
```

Q47: What is the offsetof() macro and where is it defined?

Q48: How is offsetof() macro typically implemented?

Q49: What is a Flexible Array Member (FAM) in C99?

Q50: How do you dynamically allocate memory for a structure with a Flexible Array Member?

Q51: What is the output?

```
struct Point { int x, y; };
struct Point p1 = {1, 2};
struct Point *p = &p1;
printf("%d", p->x);
```

Q52: What are bit-fields in structures?

Q53: Can you take the address of a bit-field member using &?

Q54: What is a memory alignment requirement?

Q55: What happens on CPU level during misaligned memory access?

Q56: How does member declaration order affect structure size?

Q57: What is the size of this struct on a 64-bit system?

```
struct A {
    char a;
    double b;
    int c;
};
```

Q58: How would you reorder members of struct A to minimize memory padding?

Q59: Can you copy structure values using direct pointer dereference assignment *p2 = *p1?

Q60: What dynamic memory overhead exists when allocating structures via malloc()?

Q61: What is the purpose of the container_of macro in the Linux kernel?

Q62: Write the definition of the container_of macro.

Q63: Can structures contain function pointers?

Q64: What is the output?

```
struct Node { int val; };
struct Node n = {10};
void *v = &n;
printf("%d", ((struct Node*)v)->val);
```

Q65: What is a zero-length array extension in GCC?

Q66: What is a Union in C?

Q67: What is the size of this Union?

```
union Data {
    int i;      // 4 bytes
    char c[9];  // 9 bytes
    double d;   // 8 bytes
};
```

Q68: What is Type Punning?

Q69: Is Type Punning via unions explicitly valid in C99/C11 standards?

Q70: What is the output?

```
union Convert {
    float f;
    unsigned int i;
};
union Convert c;
c.f = 1.0f;
printf("0x%X", c.i);
```

Q71: How can unions be used to check machine endianness?

Q72: What is a Tagged (Discriminated) Union?

Q73: What is the structural difference in memory allocation between struct and union?

Q74: What is the output?

```
union Test {
    int a;
    char b;
};
union Test t;
int *p1 = &t.a;
char *p2 = &t.b;
printf("%d", p1 == (int*)p2);
```

Q75: Can unions contain structures as members?

Q76: What is an anonymous union in C?

Q77: What happens when writing to one union member and reading from another?

Q78: Why is union type punning preferred over raw pointer casting?

Q79: What is the output?

```
union U {
    int arr[2];
    long long val;
};
union U u = {{10, 20}};
printf("%lld", u.val);
```

Q80: Can a union be passed by value to a function?

Q81: What is Memory-Mapped I/O (MMIO)?

Q82: Why is the volatile keyword required when mapping hardware registers to pointers?

Q83: Declare a pointer to a volatile memory-mapped 32-bit hardware register at 0x40000000.

Q84: What does volatile int * const p mean?

Q85: What does int * volatile p mean?

Q86: What is the output behavior?

```
int a = 10;
volatile int *p = &a;
int b = *p;
int c = *p;
```

Q87: What is the purpose of the mmap() POSIX system call?

Q88: What is Internal vs External Memory Fragmentation?

Q89: What is the sbrk() system call used for?

Q90: What causes a Page Fault in OS memory management?

Q91: How does the OS detect NULL pointer dereferences at hardware level?

Q92: What is a custom memory pool allocator?

Q93: What exception applies to char * regarding strict aliasing rules?

Q94: Implement a C function signature for inspecting memory byte-by-byte.

Q95: How do pointer out-of-bounds writes lead to Stack Overflow?

Q96: What is the purpose of C99 restrict pointer qualifier?

Q97: What happens if restrict promise is violated?

```
void add(int * restrict a, int * restrict b, int *val) {
    *a += *val;
    *b += *val;
}
```

Q98: What are the C11 alignment macros (_Alignas, _Alignof)?

Q99: What is Pointer Provenance in compiler optimizations?

Q100: What is a Memory Leak and how do pointers relate to it?